Pocket Vakil

A 332M-parameter small language model for Indian Penal Code statute identification

Team 15 · IAIRO / Pramana SLM++ Bootcamp · September 2026

---

1. Summary

We trained a 332M-parameter language model from scratch on 2.0 billion tokens of public Indian case law, then fine-tuned it to identify applicable Indian Penal Code sections from case facts. The task is 100-way multi-label classification: roughly 88% of cases cite more than one section.

On the full 13,019-case ILSI test split the model reaches **macro-F1 0.3585** and **micro-F1 0.4791**, with **zero of 100 sections scoring F1 = 0** and a parse-failure rate of 0.07%.

For reference, LeSICiN — the paper that introduced this dataset (Paul et al., AAAI 2022) — reports 28% macro-F1, and a 2025 model reports 0.486 micro / 0.355 macro. Evaluation setups are not identical, so treat these as a reference range rather than a leaderboard position. The comparison is nonetheless informative: a from-scratch 332M model trained in under 19 GPU-hours sits in the range of specialised systems built for this benchmark.

The model is undertrained by design constraint rather than choice. At 11.8 tokens per parameter it reached roughly 60% of the Chinchilla-optimal budget, and our own validation curve shows the second compute allocation buying almost nothing. That, not architecture, is the dominant limit on these numbers.

---

2. Task and data

The Indian Legal Statute Identification (ILSI) benchmark pairs the facts of a case with the set of IPC sections a court applied. Critically, ILSI masks statute references, act names and named entities in the fact text with <SECTION>, <ACT> and <ENTITY> tokens — measured on the first test case: 8, 9 and 61 occurrences respectively. The model cannot read answers off the input; it must infer them.

Label frequency ranges from Section 34 (3,731 cases) to Section 200 (90 cases), about 41× — a moderate imbalance rather than an extreme one, which makes zero dead labels a meaningful result rather than a lucky one.

Pretraining corpus. 1,999,624,192 tokens of public judgments from the Supreme Court and seven High Courts, packed into 976,379 sequences of length 2048.

Court	Tokens
Delhi	431M
Madras	352M
Bombay	352M
Allahabad	336M
Madhya Pradesh	290M
Supreme Court	235M
Calcutta	4.8M

Bombay and Madras were capped at 351.5M tokens to prevent single-court dominance, dropping 827,402 and 410,596 documents respectively.

---

3. Architecture

Parameters	331,964,416 total / 299.2M non-embedding
Layers	24
Model dimension	1024
Attention heads	16 (head_dim 64)
Attention	MHA, full-width q/k/v, no grouping
Positional encoding	RoPE, interleaved pairs
Normalization	LayerNorm, pre-norm
Activation	SwiGLU, ff_multiplier 2.625
Embeddings	Tied
Context	2048
Precision	bf16
Vocabulary	32,000 BPE

Built from OLM 2.2.0 primitives. The vocabulary includes 100 <ipc_*> label tokens and ILSI's three mask tokens, all present at tokenizer-training time — they cannot be added afterwards without invalidating the embedding table.

This combination matches no stock HuggingFace architecture. LayerNorm rather than RMSNorm, interleaved rather than half-split RoPE, and biases on every projection each rule out mapping onto Llama or GPT-NeoX. Section 8 covers what that meant in practice.

---

4. Training

Pretraining ran across two 20-hour H100 allocations: 10,049 steps, 3.93B tokens over 1.97 epochs of the 2.0B-token corpus. AdamW, β = (0.9, 0.999), ε = 1e-8, weight decay 0.1 on 145 tensors and 0.0 on the 242 norm scales, biases and embeddings. Gradient clipping at global norm 1.0. Effective batch 192 sequences (393,024 tokens) per step. Seed 1234.

	Slot 1	**Slot 2**
Steps	0 → 4,964	4,964 → 10,049
Peak LR	6e-4	2e-4 (restart)
Warmup	500	100
Wall clock	8h36m	8h37m
Final loss	1.4460	1.2069

The slot boundary required a scheduler restart. The restart peak of 2e-4 sits above the 1.13e-4 the run had decayed to, producing a transient loss excursion around steps 5,064–5,500. This is a deliberate schedule restart, documented in the manifest so it is not misread as instability.

Measured throughput was 64,500 tokens/s, giving MFU of roughly 13% against 989 TFLOP/s dense bf16. This sits below the ~18% our reference material reports for small models on H100 and well below the 35–45% quoted as a healthy target — consistent with small GEMMs underfilling tensor cores on a flagship card.

One reporting note worth recording: the trainer's own end-of-run summary reports 127,313 tokens/s. That figure divides cumulative tokens across both slots by slot-2-only wall clock and is inflated roughly 2×. All figures here use the corrected value.

Fine-tuning initialised from the step-10,000 checkpoint: 52,750 examples, 3 epochs, 3,294 steps, effective batch 48, peak LR 5e-5, warmup 200, cosine to 5e-6. Final loss 0.6562, 1h25m. Cross-entropy masked to response tokens only.

Total GPU consumed: **18.64 hours** for the submitted model's lineage, ~23.5 hours including discarded runs and evaluation, against 40 allocated.

---

5. What the second allocation bought

Pretraining logged no validation loss. We reconstructed a curve after the fact by running each surviving milestone checkpoint over 256 held-out ILSI fact statements.

Tokens	Val loss
0.57B	3.4536
1.15B	3.3093
1.72B	3.2388
1.95B	3.2125
2.56B	3.2249
3.15B	3.1958
3.54B	3.2021
3.74B	3.1901
3.93B	3.1895

Slot 1 delivered **−0.241 nats**. Slot 2, for the same 8.6 hours, delivered **−0.023** — about a tenth.

The reason is visible in the numbers: slot 2 was a second epoch over an unchanged corpus. We were re-reading tokens, not scaling them. The binding constraint was corpus size, not compute, and a third allocation on the same data would have bought proportionally less again.

This measurement only exists because the milestone checkpoints were still on disk when we thought to use them. They were deleted when the allocation closed a few hours later.

---

6. Results

Full ILSI test split, 13,019 cases, 100 sections:

Metric	Pocket Vakil	tf-idf baseline
micro-F1	0.4791	0.4941
macro-F1	**0.3585**	0.2858
micro-P / R	0.4670 / 0.4918	—
exact match	0.0767	—
labels with F1 = 0	**0 / 100**	—

Parse failures, measured and not repaired: 7 duplicate, 2 no-EOS, 3 non-label, **0 empty** — 0.07% overall.

Reading these numbers. Micro-F1 pools every prediction across all cases, so frequent sections dominate. Macro-F1 averages per-section F1, so a section appearing 90 times counts as much as one appearing 3,731 times. tf-idf is marginally ahead on micro and substantially behind on macro — it is better at the common sections it has abundant examples of, while the neural model spreads competence across the full label space. On a long-tailed 100-way multi-label task, that is the harder achievement, and the zero dead labels corroborate it.

Exact match of 0.0767 alongside micro-F1 of 0.4791 is diagnostic: the model frequently identifies most of a set correctly but miscounts its size. That is a signature of the generative formulation, discussed in Section 9.

Worth noting on sampling: at n=1,000 the same model showed 8 labels at F1 = 0. That was purely a sampling artifact — rare sections were not represented in the subsample. Only the full split reveals the true figure.

---

7. Ablation: fine-tuning mix

The competition rubric scores general capability and instruction-following, not our IPC task. That created an incentive to shift the fine-tuning mix toward instruction data. We tested whether it was affordable.

Both variants were fine-tuned from the identical pretrain checkpoint with identical hyperparameters, differing only in mix, and evaluated on the same 1,000 test cases.

	v2	v3
IPC / instruction / continuation	81.0 / 11.4 / 7.6	73.3 / 18.6 / 8.1

	v2	v3
micro-F1	**0.4650**	0.4494
macro-F1	**0.3470**	0.3042
labels with F1 = 0	**8**	17
micro-P / R	0.4496 / 0.4816	0.4910 / 0.4143

Reducing IPC examples from 42,750 to 30,000 cost 1.6 points of micro-F1 and 4.3 of macro, and more than doubled the dead-label count. The precision/recall split shows why: v3 became more conservative, predicting fewer sections, and the sections it stopped predicting were the rare ones.

The instruction-heavy variant was rejected. Whether it would have scored better on the rubric's instruction-following component is unmeasured — we had no harness for it — but it demonstrably degraded the task the model exists to do.

---

8. Three silent failures

Every substantial bug in this project produced clean output and raised no error. Each was caught only because a check compared against a known-good value rather than confirming that something ran.

8.1 Left-padded evaluation. The evaluation harness left-padded batches with EOS and passed no attention mask. Under causal attention, real tokens then sit *after* the pad block and attend to it, and each row's absolute positions shift by however much padding its batch happened to need.

	Before	After
micro-F1	0.2304	0.4695
parse failures	37.5%	0.2%
empty outputs	25%	0%

The bug halved the score and was invisible at batch size 1, where there is no padding. The fix: right-pad (correct without a mask, since pads follow real tokens and causality handles them), read each row's logits at its own true last position rather than the batch's, and truncate prompts to context − max_new so the generation window can never exceed context.

8.2 Uninitialised RoPE tables. Converting to HuggingFace format, we registered the rotary sin/cos tables as non-persistent buffers, matching the reference implementation. Non-persistent buffers are excluded from the saved checkpoint, and from_pretrained materialises a model *from* the checkpoint — so on reload the tables contained uninitialised memory. The cosine table began at 0.0 where it should have been 1.0.

The obvious test missed it entirely. On a single-token input the converted model matched the original to 0.000e+00, because softmax over one key returns 1.0 regardless of what the query and key contain. Divergence appeared only at sequence length ≥ 2, reaching max |Δlogit| of 18.6 with 16.7% argmax agreement.

Computing the tables in forward with a device/dtype cache leaves nothing for from_pretrained to leave uninitialised. Post-fix: **max |Δlogit| 0.000e+00, argmax agreement 100%** — bit-identical to the original.

8.3 Label namespace mismatch. The reviewer-facing evaluation script read label names from the tokenizer as <ipc_302>, while the gold labels are Section 302. They can never match. The result: zero true positives, every prediction a false positive, every gold label a false negative — micro-F1 exactly 0.0000, with **zero parse failures**. Every diagnostic said the model was generating perfectly; the scoring compared incomparable strings. The script now asserts non-empty overlap between the two label namespaces before scoring.

The common thread is that none of these raised an exception. The organisers' guidance warns that a converted model can "load fine but scoring silently break." We have three instances of exactly that, and the only thing that caught any of them was comparing against a value we independently knew to be right.

---

9. Limitations

Undertrained. 11.8 tokens per parameter against Chinchilla's 20. The validation curve in Section 5 shows returns already flattening, not because scaling failed but because the corpus was exhausted. This is the dominant limit on accuracy.

Generative label emission. The model must decide the label set, its ordering, and when to stop — three problems where a classifier has one. A sigmoid head over 100 labels would eliminate two of them for roughly 33k additional parameters. The gap between micro-F1 (0.4791) and exact match (0.0767) is this formulation showing through.

Input truncation. Inputs are capped at ~2,025 tokens. Sections cited only late in a long judgment can be missed, and the tf-idf baseline sees full documents. This is a plausible contributor to the micro-F1 gap, though we did not measure F1 by input length and cannot quantify it.

No abstention. Given out-of-distribution input the model still emits IPC sections. There is no confidence mechanism and no "none applicable" output.

Closed vocabulary. 100 sections, fixed. Anything outside is inexpressible.

General capability is low by construction. The corpus is domain-pure Indian case law — no general web text, code, mathematics or instruction data. On generic prompts the model produces fluent legalese rather than on-topic responses, and can enter repetition loops. It passes artifact-validity checks with zero empty outputs across all 13,019 test cases, but does not follow general instructions reliably.

Decontamination unverified. Explicit n-gram removal of ILSI test content from the pretraining corpus is not evidenced in our build pipeline and could not be verified after the compute node was reclaimed. Two structural factors limit leakage — ILSI's entity masking means statute references are not present verbatim, and the test split was held out of all fine-tuning and hyperparameter selection — but the pretraining corpus deliberately draws from the same High Courts ILSI is built from, so judgment-level overlap is plausible. We declare this as an unverified step rather than assert a check we cannot demonstrate.

---

10. What we would do differently

Build the eval suite before the corpus. Our own reference material states this plainly. We built it afterwards and found the padding bug on the final night, having reported a halved score for days.

Log validation loss during training. Reconstructing it from milestone checkpoints worked only because those files happened to still exist. A few hours later they did not.

Include a task marker from the first fine-tune. The initial fine-tune fed the model facts followed directly by label tokens — no separator, no instruction, no marker. With no way to distinguish "predict sections" from "continue text," after three epochs it answered everything with label tokens. On generic prompts these decode to an empty string: four of five smoke-test prompts returned empty, a hard failure of the artifact-validity gate. Reframing every example with a 23-token task frame fixed it and cost nothing on IPC accuracy — micro-F1 went from 0.4455 to 0.4650.

Use a classification head. The strongest available improvement for this task, and the cheapest.

Retain more checkpoints. The first fine-tune ran with a rolling retention of two, so no dev-set checkpoint selection was possible. Corrected in the ablation run, which retained eight.

Collect more tokens before spending the second allocation. The 8.6 hours that bought 0.023 nats would have been better spent on corpus expansion, or on the evaluation and fine-tuning work we ran out of time for.

---

11. Reproducibility

Weights and tokenizer ship as a single HuggingFace folder loadable via AutoModelForCausalLM.from_pretrained(..., trust_remote_code=True). The custom modelling code is required — no stock architecture matches this model, and mapping onto Llama or GPT-NeoX would load cleanly while computing different numbers.

Conversion from the native OLM checkpoint is verified numerically identical: max |Δlogit| = 0.000e+00, argmax agreement 100% on real ILSI input. All reported results therefore transfer to the converted model unchanged.

inference_hf.py reproduces every reported metric using only torch and transformers. The original evaluate.py is included for provenance but imports OLM 2.2.0, which was supplied inside the bootcamp environment and is not publicly installable.

One requirement a reviewer must know: **inference requires the v2 reframed test split.** The task frame is part of the model's expected input. Run against the original ILSI split, the model emits EOS on every row and scores 0.0 with no error raised.